

DAY 16

1. case Statements

What is case?

A `case` statement is useful when you want to compare **one variable** against multiple possible values or patterns.

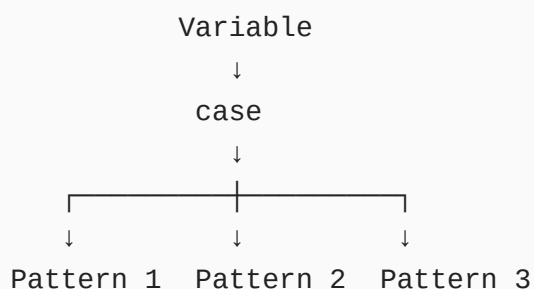
Instead of writing a long:

```
if  
elif  
elif  
elif  
else
```

you can use:

```
case "$variable" in  
    pattern1)  
        commands  
        ;;  
    pattern2)  
        commands  
        ;;  
    *)  
        default  
        ;;  
esac
```

Mental model



↓ ↓ ↓
Action 1 Action 2 Action 3

2. **case** Syntax

The general syntax is:

```
case "$variable" in
    pattern1)
        commands
        ;;
    pattern2)
        commands
        ;;
    pattern3)
        commands
        ;;
    *)
        default_commands
        ;;
esac
```

Important components

Component	Purpose
case	Starts the statement
"\$variable"	Value being checked
in	Starts the patterns
pattern)	Defines a possible match
;;	Ends a case branch
*	Default/catch-all pattern
esac	Ends the case statement

Remember :

case → esac

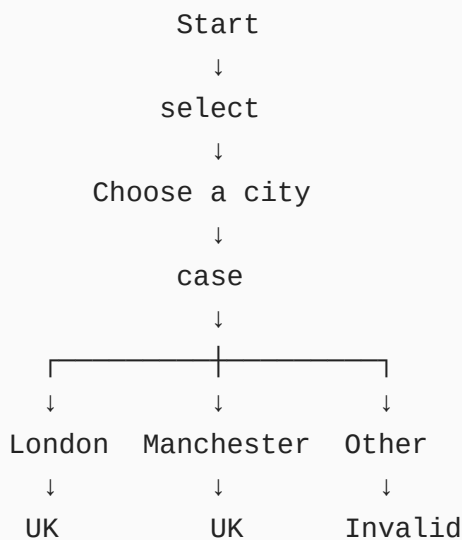
Just like:

```
if → fi
```

3. Practical Example – `city_checker.sh`

The instructor demonstrates a practical script that allows the user to select a city and then determines its country.

The overall process is:



4. Combining `select` and `case`

You previously learned about `select`.

`select` is excellent for creating a menu:

- ```
1) London
2) Manchester
3) Paris
4) Tokyo
```

The user chooses an option.

The selected value can then be handled using `case`.

Conceptually:

```
select city in London Manchester Paris Tokyo
do
 case "$city" in
 London|Manchester)
 echo "United Kingdom"
 ;;
 Paris)
 echo "France"
 ;;
 Tokyo)
 echo "Japan"
 ;;
 *)
 echo "Invalid selection"
 ;;
 esac
break
done
```

The important part is:

```
London|Manchester)
```

---

## 5. The | Symbol in case

In a `case` statement, `|` means **OR between patterns**.

So:

```
London|Manchester)
```

means:

```
London OR Manchester
```

Both can execute the same block.

For example:

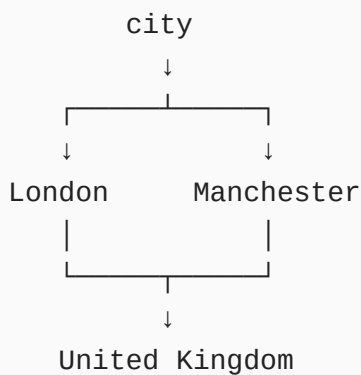
```
London|Manchester)
 echo "United Kingdom"
;;
```

This avoids repeating:

```
London)
 echo "United Kingdom"
;;

Manchester)
 echo "United Kingdom"
;;
```

## Mental model



This is one of the main advantages of `case` .

---

## 6. Default Case

Use:

```
*)
```

for inputs that don't match any specific pattern.

Example:

```
case "$city" in
 London)
```

```
 echo "United Kingdom"
 ;;
 Paris)
 echo "France"
 ;;
 *)
 echo "Unknown city"
 ;;
esac
```

Think of `*` as:

**Anything that didn't match the previous cases.**

It should normally be placed at the end.

---

## 7. `while` Loops

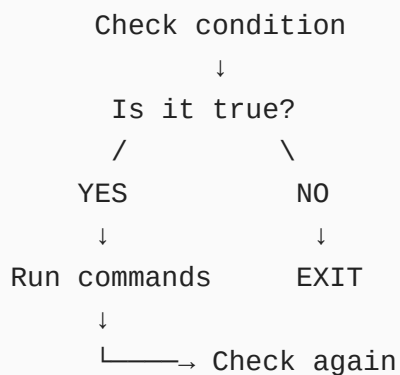
Now we move from **decision-making** to **repetition**.

A `while` loop repeatedly executes commands **as long as a condition is true**.

Basic syntax:

```
while [condition]
do
 commands
done
```

The logic is:



## 8. Simple while Example

Conceptually:

```
count=5

while ["$count" -gt 0]
do
 echo "$count"
 count=$((count - 1))
done
```

The loop works like this:

```
count = 5
 ↓
5 > 0 → YES → print 5
 ↓
count = 4
 ↓
4 > 0 → YES → print 4
 ↓
count = 3
 ↓
...
 ↓
count = 0
 ↓
0 > 0 → NO
 ↓
EXIT
```

Output:

```
5
4
3
2
1
```

---

## 9. Preventing Infinite Loops

This is **very important**.

Consider:

```
count=5

while ["$count" -gt 0]
do
 echo "$count"
done
```

What's wrong?

count never changes.

Therefore:

```
count = 5
 ↓
5 > 0 → TRUE
 ↓
print
 ↓
5 > 0 → TRUE
 ↓
print
 ↓
5 > 0 → TRUE
 ↓
...
```

The loop never ends.

That's an **infinite loop**.

---

## 10. Updating the Loop Condition

You need to change the variable controlling the condition.

For example:

```
count=$((count - 1))
```

Now:

```
5 → 4 → 3 → 2 → 1 → 0
```

Eventually:

```
count > 0
```

becomes false.

## Key rule

**A variable used in a `while` condition usually needs to change somewhere inside the loop so that the loop can eventually terminate.**

Not every `while` loop must modify a variable directly—some wait for external events or use commands whose status changes—but for counter-based loops, updating the controlling state is essential.

---

## 11. `while` + Arithmetic Expansion

You can use Bash arithmetic expansion:

```
$(expression))
```

For example:

```
count=$((count - 1))
```

This connects directly to your earlier lesson about arithmetic expansion.

You can also increment:

```
count=$((count + 1))
```

So your Bash knowledge is building on itself:

```
Variables
 ↓
Arithmetic expansion
 ↓
Conditions
 ↓
while loop
 ↓
Repeated automation
```

---

## 12. getopt

Now we get to a very useful concept for building **professional Bash command-line tools**.

Normally, a script might receive:

```
./script.sh value
```

But professional Linux utilities commonly support options:

```
ls -l
```

or:

```
command -f file
```

or:

```
command -c 25
```

Bash provides:

```
getopts
```

to help parse **short command-line options**.

---

## 13. Why `getopts` Is Useful

Suppose you're building:

```
temperature_converter.sh
```

You want users to be able to specify:

```
-f
```

for Fahrenheit and:

```
-c
```

for Celsius.

Instead of manually examining `$1`, `$2`, etc., `getopts` handles option parsing.

Conceptually:

```
./temperature_converter.sh -f 100
 |
 ↓
 getopts
 |
 ↓
option = f
argument = 100
```

---

## 14. `getopts` with `while`

A very common structure is:

```
while getopts "f:c:" option
do
 ...
done
```

The important idea is that `getopts` returns information about the next option.

It is commonly placed inside a `while` loop because the script may need to process **multiple options**.

---

## 15. What Does `"f:c:"` Mean?

Suppose you have:

```
getopts "f:c:" option
```

The letters define accepted options.

```
f:
c:
```

The colon after a letter means:

**This option requires an argument.**

So:

```
-f 100
```

means:

```
option = f
argument = 100
```

Similarly:

```
-c 25
```

means:

```
option = c
argument = 25
```

## Important

Compare:

```
f
```

with:

```
f:
```

| Specification | Meaning                         |
|---------------|---------------------------------|
| f             | -f does not require an argument |
| f:            | -f requires an argument         |

---

## 16. \$OPTARG

When an option requires an argument, Bash makes that argument available through:

```
$OPTARG
```

For example:

```
./temperature_converter.sh -f 100
```

During processing:

```
option → f
OPTARG → 100
```

So:

```
$OPTARG
```

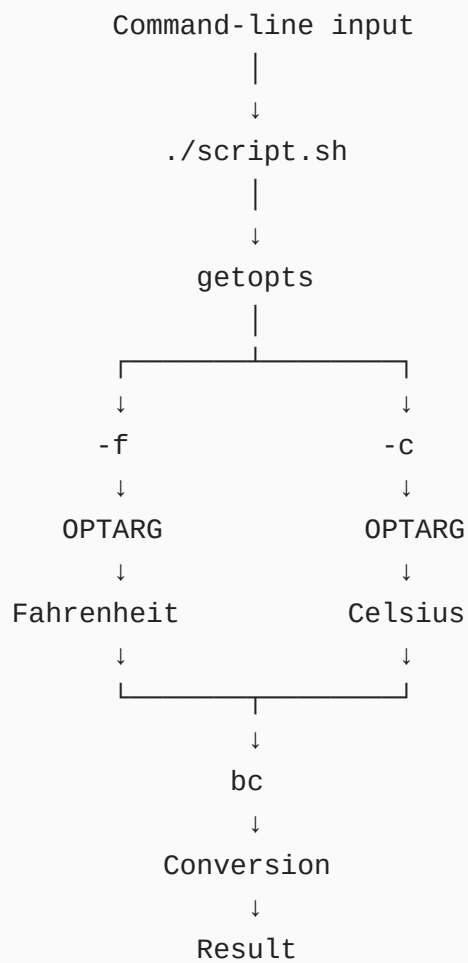
contains:

## Mental model

```
-f 100
| |
| └── OPTARG
|
└── option
```

## 21. Putting `getopts` Together

The complete conceptual architecture is:



## 22. `case` + `getopts`

These two concepts are commonly used together.

`getopts` determines **which option was provided**.

`case` determines **what to do with that option**.

Conceptually:

```
while getopts "f:c:" option
do
 case "$option" in
 f)
 # Fahrenheit conversion
 ;;
 c)
 # Celsius conversion
 ;;
 *)
 # Invalid option
 ;;
 esac
done
```

This is a very useful Bash pattern.

**Think of their jobs separately:**

`getopts` → "What option did the user give me?"

`case` → "What should I do for that option?"

---

## 23. `if` VS `case` VS `while` VS `getopts`

| Feature              | Purpose                                      |
|----------------------|----------------------------------------------|
| <code>if</code>      | Make decisions based on conditions           |
| <code>case</code>    | Select among multiple patterns/options       |
| <code>while</code>   | Repeat while a condition/status remains true |
| <code>getopts</code> | Parse short command-line options             |

A professional Bash script might use **all four**.

